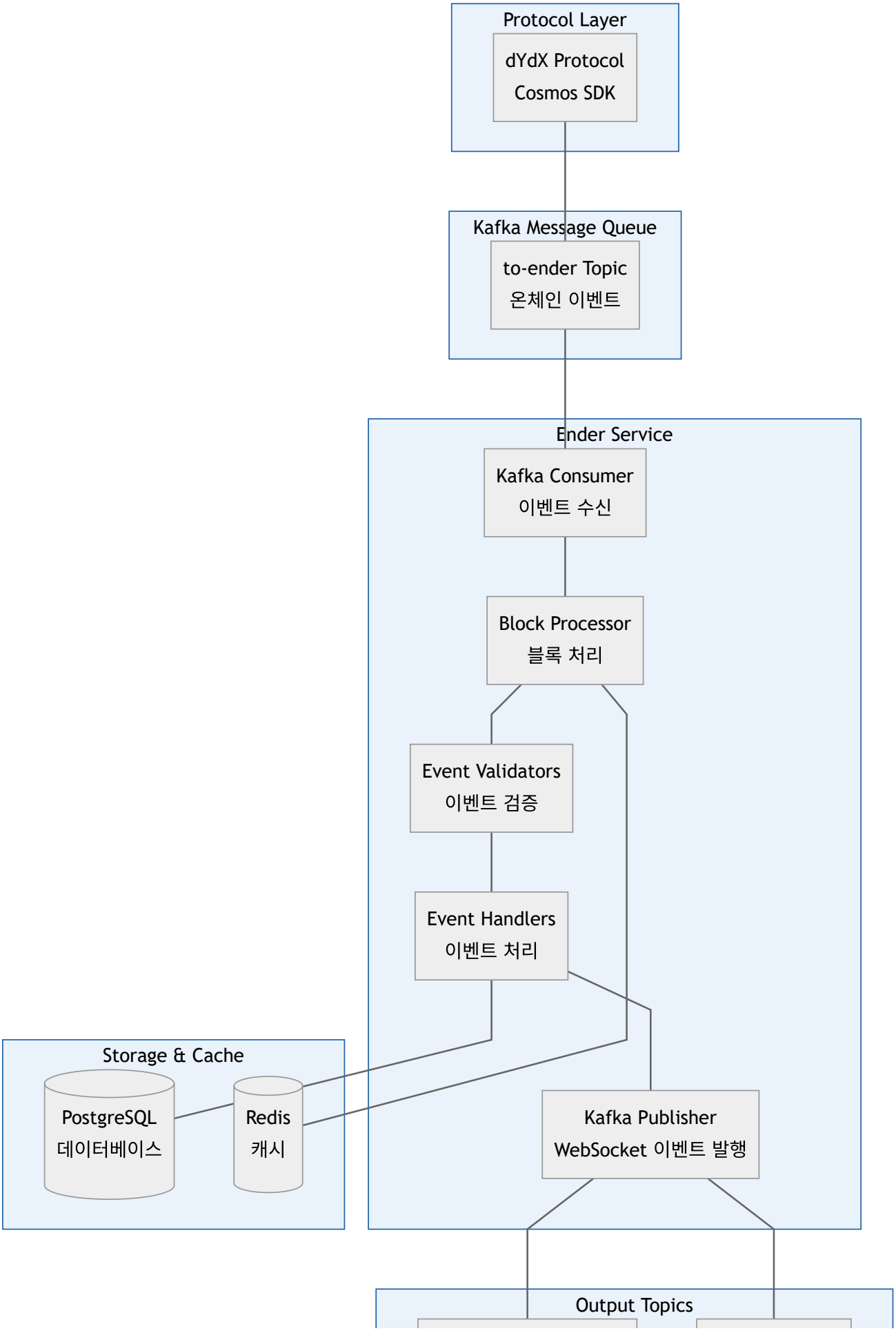
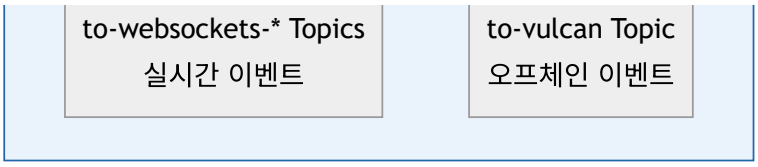
Ender 서비스 구현 분석

개요

Ender는 dYdX v4 Indexer의 핵심 서비스로, 온체인 데이터 아카이브(On-chain Data Archival) 역할을 담당합니다. Protocol에서 발생하는 모든 온체인 이벤트를 Kafka를 통해 수신하여 PostgreSQL 데이터베이스에 저장하고, 실시간 WebSocket 이벤트를 생성하여 다른 서비스들에게 전달하는 중요한 역할을 수행합니다.

아키텍처 개요


Protocol Layer
dYdX Protocol
Cosmos SDK
Kafka Message Queue
to-ender Topic
온체인 이벤트
Ender Service
Kafka Consumer
이벤트 수신
Block Processor
블록 처리
Event Validators
이벤트 검증
Event Handlers
이벤트 처리
Storage & Cache
PostgreSQL
데이터베이스
Redis
캐시
Kafka Publisher
WebSocket 이벤트 발행
Output Topics



온체인 이벤트 처리 아키텍처

1. 이벤트 수신 및 초기 처리

Ender는 `to-ender` Kafka 토픽에서 `IndexerTendermintBlock` 형태의 메시지를 수신합니다:

```
// 메인 메시지 처리 함수
export async function onMessage(message: KafkaMessage): Promise<void> {
const indexerTendermintBlock: IndexerTendermintBlock = getIndexerTendermintBlock
// 블록 중복 처리 방지
if (await shouldSkipBlock(blockHeight)) {
return;
}
// 트랜잭션 시작
const txId: number = await Transaction.start();
await Transaction.setIsolationLevel(txId, IsolationLevel.READ_UNCOMMITTED);
try {
// 블록 처리
const blockProcessor: BlockProcessor = new BlockProcessor(indexerTendermintBlock
const kafkaPublisher: KafkaPublisher = await blockProcessor.process();
// 캔들 데이터 생성
```

```
const candlesGenerator: CandlesGenerator = new CandlesGenerator(kafkaPublisher,
const candles: CandleFromDatabase[] = await candlesGenerator.updateCandles();
// 트랜잭션 커밋
await Transaction.commit(txId);
// WebSocket 메시지 발행
if (config.SEND_WEBSOCKET_MESSAGES) {
await kafkaPublisher.publish();
}
} catch (error) {
await Transaction.rollback(txId);
throw error;
}
}
```

2. 블록 처리 파이프라인

`BlockProcessor` 클래스는 다음 단계로 블록을 처리합니다:

2.1 이벤트 그룹화 (Event Grouping)

```
private groupEvents(): GroupedEvents {
const groupedEvents: GroupedEvents = {
transactionEvents: [], // 트랜잭션별 이벤트
blockEvents: [], // 블록 레벨 이벤트
```

```
};

for (const event of this.block.events) {

const transactionIndex = indexerTendermintEventToTransactionIndex(event);

const eventProtoWithType = indexerTendermintEventToEventProtoWithType(event);

if (transactionIndex === -1) {

groupedEvents.blockEvents.push(eventProtoWithType);

} else {

groupedEvents.transactionEvents[transactionIndex].push(eventProtoWithType);

}

}

return groupedEvents;

}
```

2.2 이벤트 검증 및 핸들러 조직화

```
private validateAndOrganizeEvents(groupedEvents: GroupedEvents): void {

// 트랜잭션 이벤트 처리

for (const eventsInTransaction of groupedEvents.transactionEvents) {

for (const eventProtoWithType of eventsInTransaction) {

this.validateAndAddHandlerForEvent(

eventProtoWithType,

TXN_EVENT_SUBTYPE_VERSION_TO_VALIDATOR_MAPPING

);

}
```

```
}
// 블록 이벤트 처리
for (const eventProtoWithType of groupedEvents.blockEvents) {
this.validateAndAddHandlerForEvent(
eventProtoWithType,
BLOCK_EVENT_SUBTYPE_VERSION_TO_VALIDATOR_MAPPING
);
}
}
```

3. 이벤트 타입별 처리 매핑

Ender는 다양한 이벤트 타입을 처리하며, 각각에 대해 전용 Validator와 Handler를 사용합니다:

```
const TXN_EVENT_SUBTYPE_VERSION_TO_VALIDATOR_MAPPING = {
'ORDER_FILL-1': OrderFillValidator,
'SUBACCOUNT_UPDATE-1': SubaccountUpdateValidator,
'TRANSFER-1': TransferValidator,
'MARKET-1': MarketValidator,
'STATEFUL_ORDER-1': StatefulOrderValidator,
'ASSET-1': AssetValidator,
'PERPETUAL_MARKET-1': PerpetualMarketValidator,
'LIQUIDITY_TIER-1': LiquidityTierValidator,
'UPDATE_PERPETUAL-1': UpdatePerpetualValidator,
```

```
'UPDATE_CLOB_PAIR-1': UpdateClobPairValidator,
'DELEVERAGING-1': DeleveragingValidator,
};

const BLOCK_EVENT_SUBTYPE_VERSION_TO_VALIDATOR_MAPPING = {
'FUNDING-1': FundingValidator,
'TRADING_REWARD-1': TradingRewardsValidator,
};
```

Kafka에서 이벤트 수신 및 파싱 로직

1. Kafka 연결 및 구독

```
export async function connect(): Promise<void> {
await Promise.all([
consumer.connect(),
producer.connect(),
]);

await consumer.subscribe({
topic: TO_ENDER_TOPIC,
fromBeginning: true, // 재시작 시 모든 메시지 처리
});
```

```
updateOnMessageFunction(
(_topic: string, message: KafkaMessage): Promise<void> => onMessage(message)
);
}
```

2. 메시지 파싱 및 검증

```
function getIndexerTendermintBlock(message: KafkaMessage): IndexerTendermintBloc
if (!message || !message.value || !message.timestamp) {
logger.error({ message: 'Empty message' });
return undefined;
}

try {
const messageValueBinary: Uint8Array = new Uint8Array(message.value);
const block: IndexerTendermintBlock = IndexerTendermintBlock.decode(messageValue
return block;
} catch (error) {
logger.crit({ message: 'Unable to parse message', error });
return undefined;
}
```

```
}
```

3. 이벤트 검증 시스템

각 이벤트 타입은 전용 Validator를 통해 검증됩니다:

```
export abstract class Validator<T extends object> {
event: T;
block: IndexerTendermintBlock;
blockEventIndex: number;

public abstract validate(): void;
public abstract createHandlers(
indexerTendermintEvent: IndexerTendermintEvent,
txId: number,
): Handler<EventMessage>[];
}
```

예시: SubaccountUpdateValidator

```
export class SubaccountUpdateValidator extends Validator<SubaccountUpdate> {
public validate(): void {
if (!this.event.subaccountId) {
this.logAndThrowParseMessageError('SubaccountUpdate must have a subaccountId');
}
```

```
if (!this.event.updatedAssetPositions && !this.event.updatedPerpetualPositions)
this.logAndThrowParseMessageError('SubaccountUpdate must have updated positions'
}
}
}
```

PostgreSQL 데이터베이스 저장 스키마 및 최적화

1. 데이터베이스 스키마 구조

Ender는 다음과 같은 주요 테이블에 데이터를 저장합니다:

핵심 테이블들:

- **blocks**: 블록 정보 저장
- **transactions**: 트랜잭션 정보 저장
- **tendermint_events**: 모든 Tendermint 이벤트 저장
- **fills**: 주문 체결 정보
- **orders**: 주문 정보
- **subaccounts**: 서브계정 정보
- **asset_positions**: 자산 포지션
- **perpetual_positions**: 영구 선물 포지션
- **transfers**: 자산 이전 내역
- **markets**: 시장 정보
- **candles**: 캔들스틱 데이터

2. SQL 기반 블록 처리

Ender는 PostgreSQL의 저장 프로시저를 사용하여 효율적인 블록 처리를 수행합니다:

```
const result: pg.QueryResult = await storeHelpers.rawQuery(
'SELECT dydx_block_processor(?) AS result;',
{
txId: this.txId,
bindings: [JSON.stringify(this.sqlBlock)],
sqlOptions: { name: 'dydx_block_processor' },
}
);
```

3. 데이터베이스 최적화 전략

3.1 트랜잭션 격리 수준 최적화

```
await Transaction.setIsolationLevel(txId, IsolationLevel.READ_UNCOMMITTED);
```

3.2 배치 처리 및 병렬화

- **BatchedHandlers**: 병렬 처리 가능한 이벤트들을 배치로 처리
- **SyncHandlers**: 순차 처리가 필요한 이벤트들을 동기적으로 처리

```
// Genesis 블록: 동기 → 배치 순서
if (this.block.height === 0) {
await this.syncHandlers.process(kafkaPublisher, resultRow);
```

```
await this.batchedHandlers.process(kafkaPublisher, resultRow);
} else {
// 일반 블록: 배치 → 동기 순서
await this.batchedHandlers.process(kafkaPublisher, resultRow);
await this.syncHandlers.process(kafkaPublisher, resultRow);
}
```

3.3 캐싱 전략

- **Block Cache**: 처리된 블록 추적으로 중복 처리 방지
- **Candle Cache**: 캔들스틱 데이터 캐싱
- **Market/Asset Cache**: 자주 조회되는 마켓 및 자산 정보 캐싱

```
// 블록 캐시 업데이트
updateBlockCache(blockHeight);

// 캔들 캐시 업데이트
_.forEach(candles, updateCandleCacheWithCandle);
```

WebSocket 이벤트 생성 및 전송 메커니즘

1. Kafka Publisher 아키텍처

`KafkaPublisher` 클래스는 다양한 타입의 WebSocket 이벤트를 관리합니다:

```
export class KafkaPublisher {
subaccountMessages: AnnotatedSubaccountMessage[];
tradeMessages: SingleTradeMessage[];
marketMessages: MarketMessage[];
candleMessages: CandleMessage[];
vulcanMessages: VulcanMessage[];
}
```

2. 이벤트 타입별 WebSocket 메시지 생성

2.1 서브계정 메시지

```
protected generateConsolidatedSubaccountKafkaEvent(
contents: string,
subaccountId: SubaccountId,
orderId?: string,
isFill?: boolean,
): ConsolidatedKafkaEvent {
const subaccountMessage: AnnotatedSubaccountMessage = {
blockHeight: this.block.height.toString(),
transactionIndex: indexerTendermintEventToTransactionIndex(this.indexerTendermin
eventIndex: this.indexerTendermintEvent.eventIndex,
contents,
subaccountId,
```

```
version: SUBACCOUNTS_WEBSOCKET_MESSAGE_VERSION,
orderId,
isFill,
};

return {
topic: KafkaTopics.TO_WEBSOCKETS_SUBACCOUNTS,
message: subaccountMessage,
};
}
```

2.2 거래 메시지

```
protected generateConsolidatedTradeKafkaEvent(
contents: string,
clobPairId: string,
): ConsolidatedKafkaEvent {
const tradeMessage: SingleTradeMessage = {
blockHeight: this.block.height.toString(),
transactionIndex: indexerTendermintEventToTransactionIndex(this.indexerTendermin
eventIndex: this.indexerTendermintEvent.eventIndex,
contents,
clobPairId,
```

```
version: TRADES_WEBSOCKET_MESSAGE_VERSION,

};


return {

topic: KafkaTopics.TO_WEBSOCKETS_TRADES,

message: tradeMessage,

};

}
```

3. 이벤트 집계 및 최적화

3.1 Fill 이벤트 집계

동일한 주문 ID에 대한 여러 Fill 이벤트를 하나의 메시지로 집계:

```
public aggregateFillEventsForSubaccountMessages() {

const lastEventForFills: Record<string, AnnotatedSubaccountMessage> = {};

const allFillEvents: Record<string, FillSubaccountMessageContents[]> = {};

this.subaccountMessages.forEach((message: AnnotatedSubaccountMessage) => {

if (message.isFill && message.orderId) {

const orderId: string = message.orderId;

// 모든 Fill 이벤트 수집

allFillEvents[orderId] = allFillEvents[orderId]

? allFillEvents[orderId].concat(message.subaccountMessageContents?.fills)

: message.subaccountMessageContents?.fills;
```

```
// 가장 최신 이벤트 추적

if (!lastEventForFills[orderId] ||

this.compareMessages(message, lastEventForFills[orderId]) > 0) {

lastEventForFills[orderId] = message;

}

}

});

// 최신 이벤트에 모든 Fill 정보 통합

Object.keys(lastEventForFills).forEach((orderId: string) => {

const lastEvent = lastEventForFills[orderId];

lastEvent.subaccountMessageContents!.fills = allFillEvents[orderId];

lastEvent.contents = JSON.stringify(lastEvent.subaccountMessageContents);

});

}
```

3.2 거래 메시지 그룹화

CLOB Pair별로 거래 메시지를 그룹화하여 효율성 향상:

```
public groupKafkaTradesByClobPairId(): ProducerMessage[] {

const groupedTradesMessages = _.groupBy(this.tradeMessages, 'clobPairId');

const groupedMergedTradeMessage = _.mapValues(

groupedTradesMessages,

(clobSpecificTradeMessages: SingleTradeMessage[]) => {

const tradeContents: TradeMessageContents = _.reduce(
```

```
clobSpecificTradeMessages,
(result: TradeMessageContents, currentTradeMessage: TradeMessage) => {
const contents = JSON.parse(currentTradeMessage.contents);
result.trades.push(contents.trades[0]);
return result;
},
{ trades: [] }
);


return {
blockHeight: clobSpecificTradeMessages[0].blockHeight,
contents: JSON.stringify(tradeContents),
clobPairId: clobSpecificTradeMessages[0].clobPairId,
version: TRADES_WEBSOCKET_MESSAGE_VERSION,
};
}
);
return Object.values(groupedMergedTradeMessage).map(tradeMessage => ({
value: Buffer.from(Uint8Array.from(TradeMessage.encode(tradeMessage).finish())),
}));
}
```

4. 배치 발행 시스템

```
public async publish() {
const allTopicKafkaMessages: TopicKafkaMessages[] = this.generateAllTopicKafkaMe

await Promise.all(
_.map(allTopicKafkaMessages, (topicKafkaMessages: TopicKafkaMessages) => {
const batchProducer: BatchKafkaProducer = new BatchKafkaProducer(
topicKafkaMessages.topic,
producer,
config.KAFKA_MAX_BATCH_WEBSOCKET_MESSAGE_SIZE_BYTES,
);
for (const message of topicKafkaMessages.messages) {
batchProducer.addMessageAndMaybeFlush(message);
}
return batchProducer.flush();
})
);
}
```

출력 토픽 및 메시지 형태

1. WebSocket 토픽들

- **to-websockets-subaccounts**: 서브계정 업데이트 (잔고, 포지션 변경)

- **to-websockets-trades**: 실시간 거래 내역
- **to-websockets-markets**: 시장 정보 업데이트
- **to-websockets-candles**: 캔들스틱 데이터 업데이트

2. Vulcan 토픽

- **to-vulcan**: 오프체인 이벤트 처리를 위한 메시지

3. 메시지 순서 보장

모든 메시지는 블록 높이, 트랜잭션 인덱스, 이벤트 인덱스 순으로 정렬되어 순서가 보장됩니다:

```
public sortEvents(msgs: OrderedMessage[]) {
msgs.sort((a: OrderedMessage, b: OrderedMessage) => {
if (Big(a.blockHeight).lt(b.blockHeight)) return -1;
if (Big(a.blockHeight).gt(b.blockHeight)) return 1;
if (a.transactionIndex < b.transactionIndex) return -1;
if (a.transactionIndex > b.transactionIndex) return 1;
return a.eventIndex < b.eventIndex ? -1 : 1;
});
}
```

성능 최적화 및 모니터링

1. 메트릭 수집

```
// 처리 시간 측정
stats.timing(
`${config.SERVICE_NAME}.processed_block.timing`,
Date.now() - start,
STATS_NO_SAMPLING,
{ success: success.toString() }
);

// 블록 높이 추적
stats.gauge(`${config.SERVICE_NAME}.processing_block_height`, indexerTendermintB

// 메시지 카운터
stats.increment(`${config.SERVICE_NAME}.received_kafka_message`, 1);
```

2. 에러 처리 및 복구

```
try {
// 블록 처리 로직
} catch (error) {
await Transaction.rollback(txId);
await refreshDataCaches(); // 캐시 새로고침
if (error instanceof ParseMessageError) {
```

```
logger.crit({ message: 'Parse error - V4 node bug', error });

} else {

logger.error({ message: 'Processing error', error });

}

throw error; // 메시지 재처리를 위해 에러 전파

}
```

3. 병렬화 전략

이벤트 처리 시 병렬화 ID를 통해 안전한 병렬 처리를 보장:

```
public getParallelizationIds(): string[] {

return [

`${this.eventType}_${SubaccountTable.subaccountIdToUuid(this.event.subaccountId!

`${SUBACCOUNT_ORDER_FILL_EVENT_TYPE}_${SubaccountTable.subaccountIdToUuid(this.e

];

}
```

동일한 병렬화 ID를 가진 이벤트들은 순차적으로 처리되어 데이터 일관성을 보장합니다.

결론

Ender 서비스는 dYdX v4의 온체인 데이터 처리 파이프라인의 핵심으로, 복잡한 이벤트 처리, 데이터베이스 최적화, 실시간 메시징을 통합한 고성능 아키텍처를 제공합니다. 견고한 에러 처리, 효율적인 캐싱, 그리고 확장 가능한 병렬 처리 시스템을 통해 높은 처리량과 안정성을 달성하고 있습니다.